Công Cụ Quay Màn Hình ShareX

Gemini · 40 messages · Exported November 9, 2026

YOU

Có open exe nào shift + printscr là chọn vùng rồi record rồi shift + printscr là lưu ko

GEMINI

Công cụ mã nguồn mở (.exe standalone hoặc portable) đáp ứng chính xác nhất nhu cầu này là ShareX.

Mặc định phím tắt của ShareX cho việc chọn vùng quay màn hình thành file MP4/GIF là Shift + Print Screen. Khi nhấn lần đầu, bạn kéo chọn vùng để bắt đầu quay; nhấn lại đúng tổ hợp phím Shift + Print Screen lần 2, nó sẽ lập tức dừng quay và tự động lưu file về máy.

Cách thiết lập nhanh với ShareX

1. Tải bản Portable: Tải bản Portable .zip từ trang GitHub chính thức của ShareX (ShareX/ShareX/releases), giải nén ra sẽ có file ShareX.exe chạy trực tiếp không cần cài đặt.
2. Kiểm tra phím tắt: Mở ShareX > chọn tab Hotkey settings... ở thanh bên trái. Tại mục Screen recording, phím tắt mặc định đã được gán sẵn là Shift + Print Screen.
3. Cài đặt bộ mã hóa (FFmpeg):
  • Lần đầu nhấn Shift + Print Screen, ShareX sẽ hỏi tải bộ mã hóa FFmpeg tự động (dung lượng nhẹ, tải 1 click trong vài giây).
  • Sau khi tải xong, tính năng ghi màn hình sẽ hoạt động hoàn toàn offline.
4. Cơ chế hoạt động:
  • Nhấn Shift + Print Screen $\rightarrow$ Trỏ chuột đổi thành hình chữ thập, kéo chọn vùng cần quay $\rightarrow$ Bắt đầu ghi hình (viền vùng quay chuyển sang nét đứt đỏ/xanh).
  • Nhấn Shift + Print Screen lần nữa $\rightarrow$ Dừng quay ngay lập tức và tự lưu file vào thư mục ShareX/Screenshots.

Một lựa chọn siêu nhẹ khác: ScreenToGif

Nếu bạn chỉ cần xuất ảnh động hoặc file video nhanh với một file .exe duy nhất không phụ thuộc cài đặt:

• ScreenToGif (screentogif.com - mã nguồn mở hoàn toàn, có bản Single executable/Portable).
• Vào Options $\rightarrow$ Shortcuts $\rightarrow$ đổi phím tắt Record / Pause / Stop thành Shift + PrtScn.
• Giao diện có khung hình trong suốt để căn chỉnh kích thước vùng cần quay trước khi bấm.

Bạn có muốn mình hướng dẫn chỉnh ShareX xuất thẳng ra MP4 chuẩn thay vì GIF không?

YOU

ShareX có bản siêu mini ko

GEMINI

Chính chủ nhà phát triển ShareX không có bản "Lite" hay "siêu mini" chính thức. Bản thân ShareX ôm đồm rất nhiều tính năng (OCR, upload cloud, editor, đo màu, FTP...) nên bộ source và cấu trúc cấu hình khá đồ sộ.

Nếu bạn muốn một công cụ siêu nhỏ gọn (vài MB, 1 file .exe duy nhất, mã nguồn mở, chỉ tập trung vào việc kéo chọn vùng $\rightarrow$ quay $\rightarrow$ lưu) theo đúng phím tắt bạn muốn, đây là những lựa chọn phù hợp nhất:

1. ScreenToGif (Chuẩn "siêu mini" nhất - ~4MB đến 8MB)

Dù tên là ScreenToGif, nó hỗ trợ xuất thẳng ra MP4 (dùng Windows Media Foundation hoặc FFmpeg tích hợp) cực mượt.

• Dung lượng: Bản Portable là 1 file .exe duy nhất, không cần cài đặt.
• Mã nguồn mở: 100% open-source trên GitHub.
• Thiết lập phím tắt:
  1. Mở app $\rightarrow$ biểu tượng Cài đặt (Options) $\rightarrow$ Shortcuts.
  2. Gán phím tắt Record / Pause / Stop thành Shift + PrintScreen.
  3. Khi bấm, bạn chỉ việc kéo khung viền đến vùng cần quay rồi ấn ghi; ấn lại là dừng và lưu.

2. Captura (Chuyên quay màn hình mini, siêu nhẹ)

• Mã nguồn mở: GitHub (MathewSachin/Captura).
• Dung lượng: Rất nhẹ (~4MB bản portable), ăn cực ít RAM (chỉ tầm 15–30MB khi chạy nền).
• Cách dùng:
  • Chọn chế độ Region (vùng chọn).
  • Vào phần Hotkeys gán phím tắt Record/Stop thành Shift + PrintScreen.
  • Hỗ trợ xuất MP4 bằng encoder phần cứng (NVENC/Intel QSV) hoặc H.264 mặc định của Windows mà không cần tải thêm FFmpeg ngoài nếu không muốn. (Lưu ý: Dự án này hiện đã ngưng cập nhật tính năng mới nhưng bản Portable v9.0 vẫn chạy cực kỳ ổn định trên Windows 10/11).

3. Nếu vẫn muốn dùng ShareX dạng "tinh giản" (Headless)

Nếu đã quen với bộ đo khung chữ thập mượt mà của ShareX và muốn biến nó thành "mini":

• Dùng ShareX Portable (.zip).
• Vào Application settings:
  • Tích chọn: Start with Windows và Minimize to tray.
  • Tắt hết các thông báo (Notifications), tắt âm thanh capture.
  • Bỏ hết các tính năng upload (chọn Destination là None/Local file only).
• Lúc này ShareX chỉ chạy ngầm dưới System Tray, ăn khoảng ~30–40MB RAM và chỉ phản hồi khi bạn ấn Shift + Print Screen để chọn vùng và quay.

YOU

Nên lấy nguồn nào gán mặc định r rebuild lại?

GEMINI

Để tối ưu cho việc fork về rebuild một bản “siêu mini” (chỉ có tính năng: bấm phím tắt $\rightarrow$ kéo chọn vùng $\rightarrow$ gọi FFmpeg/MediaFoundation quay $\rightarrow$ bấm lại lưu thẳng .mp4), nguồn phù hợp nhất phụ thuộc vào stack bạn muốn dùng:

Lựa chọn 1: Fork ShareX (C# / .NET) — Giữ trải nghiệm chọn vùng mượt nhất

Nếu bạn muốn giữ nguyên cơ chế crosshair (thước ngắm chữ thập) bắt cạnh cửa sổ cực chuẩn và pipeline xử lý FFmpeg có sẵn:

• Repo: ShareX/ShareX
• Vị trí code cốt lõi cần giữ lại:
  • ShareX.ScreenCaptureLib: Toàn bộ logic vẽ overlay trong suốt, bắt tọa độ vùng chọn kéo thả chuột.
  • ShareX.MediaLib: Module đóng gói gọi FFmpeg CLI chạy ngầm với các tham số pipe (-f gdigrab hoặc -f dshow).
  • ShareX.HelpersLib: Bộ đăng ký Global Hotkey (bắt Shift + PrintScreen toàn hệ thống).
• Những phần cần cắt bỏ khi rebuild:
  • Xóa toàn bộ các project ShareX.UploadersLib (Imgur, Drive, FTP,...).
  • Xóa các form UI cấu hình phức tạp, OCR, History window, image editor.
  • Chỉ để lại một file thực thi chạy nền dưới System Tray hoặc chạy headless với một file .ini/.json cấu hình đường dẫn FFmpeg và thư mục output.

Lựa chọn 2: Fork Captura (C# / WPF) — Dễ bóc tách nhất

Captura là dự án được thiết kế dạng module rất sạch, code ghi hình tách biệt hoàn toàn với UI, nên việc thu gọn thành bản mini dễ hơn ShareX rất nhiều.

• Repo: MathewSachin/Captura
• Vị trí code cốt lõi:
  • Captura.Core: Quản lý tiến trình quay, bắt frame, encoder wrapper.
  • Captura.Windows: Chứa logic capture qua Windows Graphics Capture API / Desktop Duplication API (hiệu năng cao hơn GDI grab truyền thống).
• Ưu điểm khi rebuild:
  • Hỗ trợ sẵn Windows Media Foundation (ghi thẳng H.264 qua phần cứng không bắt buộc phải bundle theo file ffmpeg.exe).
  • Không bị dính nhiều dependency râu ria về upload lên mạng như ShareX.

Lựa chọn 3: Tự build mới từ template C++ / Rust (Siêu nhẹ, < 2MB, Native)

Nếu mục tiêu của bạn là file .exe độc lập siêu nhẹ, không phụ thuộc .NET Runtime:

1. Global Hotkey: Đăng ký hàm Win32 API RegisterHotKey(NULL, 1, MOD_SHIFT, VK_SNAPSHOT);.
2. Region Overlay: Tạo 1 cửa sổ trong suốt không viền (WS_EX_LAYERED | WS_EX_TRANSPARENT), bắt sự kiện kéo chuột WM_LBUTTONDOWN và WM_LBUTTONUP để lấy tọa độ (x, y, width, height).
3. Recording Engine:
  • Đơn giản nhất: Chạy ngầm tiến trình ffmpeg.exe qua lệnh:

```bash
    ffmpeg -f gdigrab -framerate 30 -offset_x [X] -offset_y [Y] -video_size [W]x[H] -i
desktop -c:v libx264 -preset ultrafast -crf 23 output.mp4
```

• Khi bấm phím tắt lần 2: Gửi tín hiệu q vào stdin của tiến trình FFmpeg để dừng ghi và đóng gói file MP4 an toàn.

Tóm lại nên chọn nguồn nào?

• Nếu muốn tái sử dụng UI chọn vùng đẹp, chính xác: Chọn ShareX (bóc module ShareX.ScreenCaptureLib).
• Nếu muốn code tinh gọn, dễ đọc, dễ compile ngay: Chọn Captura.
• Nếu muốn dung lượng tối thiểu tuyệt đối: Tự viết 1 wrapper Win32 C++/C# gọi ffmpeg.exe hoặc Windows

Graphics Capture API.

YOU

Tự build mới thì nhanh ko?

GEMINI

Tự build mới một tool như vậy rất nhanh — chỉ mất tầm 15 đến 30 phút nếu viết bằng C# (.NET WinForms/WPF) dạng single-file hoặc script wrapper, vì toàn bộ logic nặng (encode video, mux file MP4) đã do tiến trình ffmpeg.exe ngầm xử lý. Bạn không cần phải code bộ giải mã/mã hóa video từ đầu.

Cấu trúc chương trình chỉ cần 3 khối logic

```plaintext
[Global Hotkey] (Shift + PrtScn)

        Trạng thái    1: Chưa quay
             Hiện Form Overlay mờ toàn màn hình -> Chuột kéo chọn Box -> Chạy ngầm
ffmpeg.exe

        Trạng thái    2: Đang quay
              Gửi phím   'q' vào tiến trình ffmpeg -> Đóng gói MP4 an toàn -> Xong
```

1. Global Hotkey (Win32 API): Gọi RegisterHotKey với MOD_SHIFT (0x0004) và VK_SNAPSHOT (0x2C).
2. Form chọn vùng (Snipping Overlay): Một cửa sổ Windows không viền, phủ toàn bộ màn hình với độ mờ (Opacity = 0.3), bắt 2 sự kiện MouseDown và MouseUp để tính ra X, Y, Width, Height.
3. Gọi FFmpeg: Kích hoạt ffmpeg.exe ẩn cửa sổ (CreateNoWindow = true, UseShellExecute = false) với RedirectStandardInput = true. Khi bấm phím lần 2, ghi ký tự q vào stream input để FFmpeg kết thúc file MP4 hoàn chỉnh.

Toàn bộ mã nguồn hoàn chỉnh (C# Single File - Program.cs)

Bạn có thể tạo một project WinForms nhỏ bằng lệnh CLI:

```bash
dotnet new winforms -n MiniRecorder
```

Sau đó thay toàn bộ file Program.cs bằng đoạn mã sau:

```c#
using System;
using System.Diagnostics;
using System.Drawing;
using System.IO;
using System.Runtime.InteropServices;
using System.Windows.Forms;

static class Program
{
    private const int HOTKEY_ID = 9001;
    private const uint MOD_SHIFT = 0x0004;
    private const uint VK_SNAPSHOT = 0x2C; // Phím PrintScreen

    private static Process? ffmpegProcess = null;
    private static HotkeyWindow? hotkeyWnd;
```

```
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        hotkeyWnd = new HotkeyWindow();
        hotkeyWnd.HotkeyPressed += OnHotkeyPressed;
        RegisterHotKey(hotkeyWnd.Handle, HOTKEY_ID, MOD_SHIFT, VK_SNAPSHOT);

        Application.Run();

        UnregisterHotKey(hotkeyWnd.Handle, HOTKEY_ID);
    }

    private static void OnHotkeyPressed()
    {
        // Nếu đang quay -> Bấm phím để dừng
        if (ffmpegProcess != null && !ffmpegProcess.HasExited)
        {
            StopRecording();
            return;
        }

        // Nếu chưa quay -> Mở lớp phủ để chọn vùng
        var snipper = new SnippingOverlay();
        if (snipper.ShowDialog() == DialogResult.OK && snipper.SelectedRegion.Width > 10 &&
snipper.SelectedRegion.Height > 10)
        {
            StartRecording(snipper.SelectedRegion);
        }
    }

    private static void StartRecording(Rectangle rect)
    {
        // FFmpeg yêu cầu kích thước khung hình phải là số chẵn
        int w = rect.Width - (rect.Width % 2);
        int h = rect.Height - (rect.Height % 2);
        int x = rect.X;
        int y = rect.Y;

        string timestamp = DateTime.Now.ToString("yyyyMMdd_HHmmss");
        string outputPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
$"rec_{timestamp}.mp4");

        // Lệnh quay màn hình gdigrab cực nhẹ
        string args = $"-y -f gdigrab -framerate 30 -offset_x {x} -offset_y {y} -video_size
{w}x{h} -i desktop -c:v libx264 -preset ultrafast -crf 23 -pix_fmt yuv420p
\"{outputPath}\"";

        var psi = new ProcessStartInfo
        {
            FileName = "ffmpeg.exe",
            Arguments = args,
            UseShellExecute = false,
            CreateNoWindow = true,
            RedirectStandardInput = true
        };

        ffmpegProcess = Process.Start(psi);
    }

    private static void StopRecording()
```

```
    {
        if (ffmpegProcess != null && !ffmpegProcess.HasExited)
        {
            try
            {
                // Gửi 'q' vào stdin để ffmpeg đóng MP4 đúng chuẩn
                ffmpegProcess.StandardInput.WriteLine("q");
                ffmpegProcess.WaitForExit(3000);
            }
            catch { }
            finally
            {
                ffmpegProcess.Dispose();
                ffmpegProcess = null;
            }
        }
    }

    [DllImport("user32.dll")]
    private static extern bool RegisterHotKey(IntPtr hWnd, int id, uint fsModifiers, uint
vk);

    [DllImport("user32.dll")]
    private static extern bool UnregisterHotKey(IntPtr hWnd, int id);
}

// Cửa sổ ngầm để bắt thông điệp phím nóng
public class HotkeyWindow : NativeWindow
{
    private const int WM_HOTKEY = 0x0312;
    public event Action? HotkeyPressed;

    public HotkeyWindow()
    {
        CreateHandle(new CreateParams());
    }

    protected override void WndProc(ref Message m)
    {
        if (m.Msg == WM_HOTKEY)
        {
            HotkeyPressed?.Invoke();
        }
        base.WndProc(ref m);
    }
}

// Lớp phủ màn hình để kéo chọn toạ độ
public class SnippingOverlay : Form
{
    public Rectangle SelectedRegion { get; private set; }
    private Point startPoint;
    private bool isDragging = false;

    public SnippingOverlay()
    {
        FormBorderStyle = FormBorderStyle.None;
        WindowState = FormWindowState.Maximized;
        TopMost = true;
        Cursor = Cursors.Cross;
        BackColor = Color.Black;
```

```
        Opacity = 0.25;
        DoubleBuffered = true;
    }

    protected override void OnMouseDown(MouseEventArgs e)
    {
        if (e.Button == MouseButtons.Left)
        {
            isDragging = true;
            startPoint = e.Location;
        }
    }

    protected override void OnMouseMove(MouseEventArgs e)
    {
        if (isDragging)
        {
            int x = Math.Min(startPoint.X, e.X);
            int y = Math.Min(startPoint.Y, e.Y);
            int width = Math.Abs(startPoint.X - e.X);
            int height = Math.Abs(startPoint.Y - e.Y);
            SelectedRegion = new Rectangle(x, y, width, height);
            Invalidate();
        }
    }

    protected override void OnMouseUp(MouseEventArgs e)
    {
        if (isDragging)
        {
            isDragging = false;
            DialogResult = DialogResult.OK;
            Close();
        }
    }

    protected override void OnKeyDown(KeyEventArgs e)
    {
        if (e.KeyCode == Keys.Escape)
        {
            DialogResult = DialogResult.Cancel;
            Close();
        }
    }

    protected override void OnPaint(PaintEventArgs e)
    {
        if (SelectedRegion.Width > 0 && SelectedRegion.Height > 0)
        {
            using var pen = new Pen(Color.Red, 2);
            e.Graphics.DrawRectangle(pen, SelectedRegion);
        }
    }
}
```

Lưu ý thực tế khi tự build

• Kích thước số chẵn: FFmpeg với codec H.264 bắt buộc Width và Height phải là bội số của 2 (đã được xử lý ở dòng rect.Width - (rect.Width % 2) trong code).
• Đóng file an toàn: Không được gọi Process.Kill(), vì video MP4 cần ghi đoạn metadata moov atom ở cuối

file. Gửi lệnh q qua StandardInput giúp file video không bao giờ bị lỗi hỏng (corrupt).
• File đi kèm: Chỉ cần ném 1 file ffmpeg.exe (bản essentials build chỉ vài chục MB) cùng thư mục với file .exe vừa build là chạy được ở bất kỳ máy nào mà không cần cài đặt.

YOU

Viết từ python ổn ko

GEMINI

Dùng Python để viết tool này hoàn toàn ổn và chạy rất tốt, thậm chí viết code còn ngắn hơn C# vì hệ sinh thái thư viện xử lý hotkey và GUI của Python rất sẵn.

Tuy nhiên, có 2 điểm khác biệt thực tế bạn cần cân nhắc trước khi chốt:

Điểm mạnh & Điểm hạn chế của Python

• Ưu điểm:
  • Code cực ngắn (tầm 70–80 dòng là xong).
  • Viết xong test được ngay không cần setup compiler/SDK rườm rà.
  • Tận dụng thư viện keyboard (hook global key cực nhạy) và tkinter (có sẵn trong Python, không cần cài thêm).
• Nhược điểm (khi đóng gói thành .exe):
  • Dung lượng file .exe lớn: Dùng PyInstaller đóng gói thì riêng file .exe nền Python đã nặng khoảng 15MB – 25MB (chưa kể file ffmpeg.exe), trong khi C# AOT chỉ tầm vài MB.
  • Tốc độ khởi động (Cold Start): File .exe đóng gói từ Python khi chạy lên sẽ mất khoảng 0.5 – 1.5 giây để giải nén runtime ra thư mục tạm. Dù vậy, vì tool này chạy nền liên tục dưới system tray/background để chờ bấm phím nên nhược điểm này không ảnh hưởng đến trải nghiệm lúc bấm quay.

Code Python hoàn chỉnh (main.py)

Chỉ cần cài thêm đúng 1 thư viện bắt phím:

bash
```
pip install keyboard
```

Tạo file main.py đặt cùng thư mục với ffmpeg.exe:

python
```
import subprocess
import datetime
import os
import sys
import tkinter as tk
import keyboard

ffmpeg_process = None

def get_output_path():
    base_dir = os.path.dirname(os.path.abspath(sys.argv[0]))
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    return os.path.join(base_dir, f"rec_{timestamp}.mp4")

def start_recording(x, y, w, h):
    global ffmpeg_process
```

```
    # FFmpeg H.264 yêu cầu width và height là số chẵn
    w = w - (w % 2)
    h = h - (h % 2)
    if w <= 0 or h <= 0:
        return

    output_file = get_output_path()

    cmd = [
        "ffmpeg.exe",
        "-y",
        "-f", "gdigrab",
        "-framerate", "30",
        "-offset_x", str(x),
        "-offset_y", str(y),
        "-video_size", f"{w}x{h}",
        "-i", "desktop",
        "-c:v", "libx264",
        "-preset", "ultrafast",
        "-crf", "23",
        "-pix_fmt", "yuv420p",
        output_file
    ]

    # Khởi chạy ẩn cửa sổ cmd, redirect stdin để gửi lệnh dừng
    creation_flags = subprocess.CREATE_NO_WINDOW if sys.platform == "win32" else 0
    ffmpeg_process = subprocess.Popen(
        cmd,
        stdin=subprocess.PIPE,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        creationflags=creation_flags
    )

def stop_recording():
    global ffmpeg_process
    if ffmpeg_process and ffmpeg_process.poll() is None:
        try:
            # Gửi phím 'q' vào stdin để FFmpeg finalize metadata moov atom
            ffmpeg_process.communicate(input=b"q", timeout=3)
        except Exception:
            ffmpeg_process.kill()
        finally:
            ffmpeg_process = None

class SnippingOverlay:
    def __init__(self):
        self.root = tk.Tk()
        self.root.attributes("-fullscreen", True)
        self.root.attributes("-alpha", 0.3)
        self.root.attributes("-topmost", True)
        self.root.config(bg="black", cursor="cross")

        self.canvas = tk.Canvas(self.root, cursor="cross", bg="grey10",
highlightthickness=0)
        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.start_x = 0
        self.start_y = 0
        self.rect = None
        self.coords = None
```

```
        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_drag)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)
        self.root.bind("<Escape>", lambda e: self.root.destroy())

    def on_press(self, event):
        self.start_x = event.x
        self.start_y = event.y
        self.rect = self.canvas.create_rectangle(self.start_x, self.start_y, self.start_x,
self.start_y, outline="red", width=2)

    def on_drag(self, event):
        cur_x, cur_y = event.x, event.y
        self.canvas.coords(self.rect, self.start_x, self.start_y, cur_x, cur_y)

    def on_release(self, event):
        end_x, end_y = event.x, event.y
        x1 = min(self.start_x, end_x)
        y1 = min(self.start_y, end_y)
        w = abs(end_x - self.start_x)
        h = abs(end_y - self.start_y)
        if w > 10 and h > 10:
            self.coords = (x1, y1, w, h)
        self.root.destroy()

    def get_selection(self):
        self.root.mainloop()
        return self.coords

def toggle_recording():
    global ffmpeg_process
    # Nếu đang quay -> Bấm lần nữa để dừng
    if ffmpeg_process and ffmpeg_process.poll() is None:
        stop_recording()
        return

    # Nếu chưa quay -> Mở overlay chọn vùng
    overlay = SnippingOverlay()
    region = overlay.get_selection()
    if region:
        x, y, w, h = region
        start_recording(x, y, w, h)

if __name__ == "__main__":
    # Đăng ký tổ hợp Shift + Print Screen
    keyboard.add_hotkey("shift+print screen", toggle_recording)
    # Giữ script chạy nền chờ hotkey
    keyboard.wait()
```

Đóng gói thành file .exe duy nhất (không hiện console đen)

Chạy lệnh PyInstaller:

```bash
pip install pyinstaller
pyinstaller --noconsole --onefile main.py
```

• File tạo ra nằm trong thư mục dist/main.exe.
• Đặt main.exe nằm chung với ffmpeg.exe.

• Khi click đúp main.exe, nó sẽ chạy hoàn toàn ngầm không hiện cửa sổ console. Bấm Shift + Print Screen lần đầu để quét chuột, bấm lại lần nữa là có file .mp4 xuất hiện cạnh đó.

Bạn có muốn thêm icon khay hệ thống (System Tray) để dễ bấm thoát script khi không dùng nữa không?

YOU

Taskx dùng ffmpeg?

GEMINI

Nếu ý bạn là ShareX (có thể bị gõ nhầm thành Taskx), thì đúng, ShareX sử dụng chính FFmpeg để thực hiện toàn bộ việc quay video màn hình (xuất ra MP4, WebM, AVI) và ảnh GIF.

Cụ thể cách nó hoạt động:

• Lần đầu tiên bạn nhấn quay (Shift + Print Screen), ShareX sẽ tự động tải file ffmpeg.exe về thư mục của nó.
• Khi quay, ShareX chạy ngầm ffmpeg.exe với module chụp màn hình của Windows (thường là qua screen-capture-recorder hoặc gdigrab), sau đó encode trực tiếp theo thiết lập FPS và bitrate bạn chọn.

Nếu "Taskx" bạn đang nhắc đến là một phần mềm hoặc thư viện khác, bạn có thể gửi thêm ngữ cảnh hoặc link để mình đối chiếu chính xác nhé!

YOU

Checklist requirements

GEMINI

Yêu cầu kiểm thử & Hoàn thiện Mini Screen Recorder

YOU

Shift printscr là mặv định hầu hết mọi app quay?

GEMINI

Không, Shift + PrintScreen không phải là phím tắt mặc định phổ biến của hầu hết các phần mềm quay màn hình.

Tổ hợp phím này gần như là "đặc sản" riêng của hệ sinh thái ShareX. Đa số các phần mềm và hệ điều hành khác đều dùng các bộ phím hoàn toàn khác:

Phần mềm / Hệ thống	Phím tắt mặc định cho Quay màn hình	Hành vi mặc định
ShareX	Shift + Print Screen	Quay vùng màn hình (Screen recording)
Windows Snipping Tool (Win 11)	Win + Shift + S (chuyển tab camera) hoặc Win + Alt + R	Chụp vùng / Quay cửa sổ
Xbox Game Bar (Windows tích hợp)	Win + Alt + R	Bắt đầu / Dừng quay toàn màn hình ứng dụng
OBS Studio	Không gán sẵn	Người dùng tự đặt trong Settings Hotkeys
Bandicam	F12	Bắt đầu / Dừng ghi hình

OBS / ShadowPlay (NVIDIA)	Alt + F9	Bật / Tắt ghi hình thủ công
AMD Radeon ReLive	Ctrl + Shift + R	Bắt đầu / Dừng ghi hình
macOS	Cmd + Shift + 5	Mở thanh công cụ chọn vùng quay/chụp

Vai trò chuẩn của nút Print Screen trên Windows

Theo quy chuẩn của hệ điều hành Windows:

• Print Screen: Chụp toàn bộ màn hình vào Clipboard.
• Alt + Print Screen: Chỉ chụp cửa sổ đang active (cửa sổ đang mở và thao tác).
• Win + Print Screen: Chụp toàn màn hình và tự lưu file ảnh vào thư mục Pictures\Screenshots.
• Shift + Print Screen: Trong Windows gốc, tổ hợp này không có chức năng riêng biệt (hệ điều hành chỉ nhận diện như một lệnh chụp màn hình thông thường).

Chính vì Shift + Print Screen bị Windows "bỏ trống", ShareX đã chọn nó làm phím tắt mặc định để tránh xung đột với các ứng dụng khác.

Bạn có muốn đổi sang một tổ hợp phím thông dụng hơn như Win + Shift + R không?

YOU

C# là native nhất?

GEMINI

C# không phải là native nhất. Về mặt kỹ thuật thuần túy, C và C++ mới là những ngôn ngữ "native" tuyệt đối trên Windows vì chúng được biên dịch thẳng ra mã máy x86/x64, gọi trực tiếp các API hạt nhân của hệ điều hành mà không cần bất kỳ môi trường thực thi (runtime) trung gian nào.

Tuy nhiên, với một tool nhỏ như trình quay màn hình bạn đang làm, C# thường được coi là lựa chọn cân bằng tốt nhất giữa hiệu năng native và tốc độ code.

So sánh mức độ "Native" trên Windows

Ngôn ngữ	Kiểu thực thi	Mức độ phụ thuộc Runtime	Dung lượng file .exe	Khả năng tương tác Windows API
C / C++	Mã máy thuần túy	Không phụ thuộc (chỉ cần các file .dll có sẵn của Windows)	Cực nhỏ (< 1 MB)	Gọi trực tiếp Win32 API gốc
Rust	Mã máy thuần túy	Không phụ thuộc	Rất nhỏ (1 – 3 MB)	Gọi trực tiếp qua winapi/windows-rs
C# (Mặc định)	Mã trung gian IL	Cần .NET Runtime (máy tính phải cài trước)	Vài chục đến vài trăm KB	Gọi Win32 qua P/Invoke ([DllImport])
C# (Native AOT)	Mã máy thuần túy	Đã đóng gói GC tối giản vào file	5 – 10 MB (single file)	Gọi Win32 qua P/Invoke
Python	Trình thông dịch	Phụ thuộc Python core + thư viện bên ngoài	20 – 35 MB (khi pack qua PyInstaller)	Phải bọc qua ctypes hoặc thư viện hook

Tại sao nhiều người vẫn nghĩ C# là native nhất trên Windows?

1. Hệ sinh thái chính chủ Microsoft: C# được Microsoft tạo ra để viết ứng dụng Windows. Toàn bộ framework giao diện (WinForms, WPF, WinUI) đều được tối ưu sâu cho kiến trúc Windows.
2. Windows cài sẵn .NET Framework: Trên mọi bản Windows 10/11 hiện nay, Microsoft đều tích hợp sẵn .NET Framework 4.8. Nếu bạn compile một project C# nhắm mục tiêu vào .NET Framework 4.8, file .exe sinh

ra chỉ nặng tầm 100KB – 200KB và có thể click đúp chạy ngay trên bất kỳ máy Windows nào mà không bắt người dùng cài thêm gì.
3. Cơ chế Native AOT (.NET Core 8/9): C# hiện đại đã hỗ trợ tính năng Native AOT (Ahead-Of-Time compilation). Khi bật cờ này, trình biên dịch sẽ chuyển toàn bộ code C# thành mã máy thuần túy giống hệt C++, loại bỏ việc nén runtime giải nén chậm chạp như Python.

Tóm lại: Bạn nên chọn gì?

• Nếu muốn native thực thụ (nhẹ nhất, ăn ít RAM nhất, khởi động ngay tức thì): Chọn C++ (dùng Win32 API thuần, không framework nặng).
• Nếu muốn code nhanh, giao diện Form bắt chuột dễ làm, file gọn: Chọn C# (.NET Framework 4.8). Đây là cách cân bằng nhất: code chỉ 100 dòng như ví dụ trước, file build ra chỉ vài trăm KB, chạy mượt mà không có độ trễ.
• Nếu muốn thử nghiệm nhanh nhất mà không cần cài Visual Studio: Dùng Python, chấp nhận file .exe nặng hơn khoảng 20MB.

YOU

Viết c++ cho app này ko?

GEMINI

Viết bằng C++ thuần Win32 API cho ứng dụng này hoàn toàn khả thi và là lựa chọn tối ưu nhất nếu muốn:

• File .exe xuất xưởng siêu nhẹ (chỉ khoảng 50 KB – 100 KB).
• Chạy trực tiếp, không phụ thuộc .NET Runtime hay môi trường Python.
• Khởi động tức thì trong 0 giây, ăn dưới 5 MB RAM khi chạy ngầm.

Mã nguồn C++ hoàn chỉnh (main.cpp)

Toàn bộ ứng dụng được gói gọn trong 1 file C++ sử dụng Win32 API gốc để đăng ký Global Hotkey, tạo Overlay kéo chọn vùng mờ trong suốt và điều khiển tiến trình ffmpeg.exe qua Pipe ẩn.

```c++
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <string>
#include <ctime>
#include <algorithm>

#define HOTKEY_ID 9001

// Global state
HANDLE g_hFFmpegStdinWrite = NULL;
PROCESS_INFORMATION g_ffmpegPi = { 0 };
bool g_isRecording = false;

// Overlay selection state
bool g_isDragging = false;
POINT g_ptStart = { 0, 0 };
RECT g_selectedRect = { 0, 0, 0, 0 };
bool g_selectionConfirmed = false;

void StartRecording(int x, int y, int w, int h) {
    // Ép kích thước về số chẵn cho chuẩn H.264
    w -= (w % 2);
```

```
    h -= (h % 2);
    if (w <= 0 || h <= 0) return;

    // Tạo timestamp cho tên file
    time_t rawtime;
    struct tm timeinfo;
    char timeStr[64];
    time(&rawtime);
    localtime_s(&timeinfo, &rawtime);
    strftime(timeStr, sizeof(timeStr), "%Y%m%d_%H%M%S", &timeinfo);

    char currentDir[MAX_PATH];
    GetModuleFileNameA(NULL, currentDir, MAX_PATH);
    char* lastSlash = strrchr(currentDir, '\\');
    if (lastSlash) *(lastSlash + 1) = '\0';

    std::string outputPath = std::string(currentDir) + "rec_" + timeStr + ".mp4";

    // Setup Anonymous Pipe để gửi lệnh 'q' dừng an toàn
    SECURITY_ATTRIBUTES sa;
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.bInheritHandle = TRUE;
    sa.lpSecurityDescriptor = NULL;

    HANDLE hStdinRead = NULL;
    if (!CreatePipe(&hStdinRead, &g_hFFmpegStdinWrite, &sa, 0)) return;
    SetHandleInformation(g_hFFmpegStdinWrite, HANDLE_FLAG_INHERIT, 0);

    STARTUPINFOA si;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.hStdInput = hStdinRead;
    si.dwFlags |= STARTF_USESTDHANDLES;

    ZeroMemory(&g_ffmpegPi, sizeof(g_ffmpegPi));

    std::string cmd = "ffmpeg.exe -y -f gdigrab -framerate 30 -offset_x " +
std::to_string(x) +
        " -offset_y " + std::to_string(y) + " -video_size " + std::to_string(w) + "x" +
std::to_string(h) +
        " -i desktop -c:v libx264 -preset ultrafast -crf 23 -pix_fmt yuv420p \"" +
outputPath + "\"";

    BOOL success = CreateProcessA(
        NULL,
        &cmd[0],
        NULL,
        NULL,
        TRUE,
        CREATE_NO_WINDOW,
        NULL,
        currentDir,
        &si,
        &g_ffmpegPi
    );

    CloseHandle(hStdinRead);

    if (success) {
        g_isRecording = true;
    } else {
```

```
        CloseHandle(g_hFFmpegStdinWrite);
        g_hFFmpegStdinWrite = NULL;
    }
}

void StopRecording() {
    if (g_isRecording && g_hFFmpegStdinWrite != NULL) {
        DWORD bytesWritten;
        const char* q = "q\n";
        WriteFile(g_hFFmpegStdinWrite, q, (DWORD)strlen(q), &bytesWritten, NULL);

        // Chờ FFmpeg ghi metadata moov atom tối đa 3 giây
        WaitForSingleObject(g_ffmpegPi.hProcess, 3000);

        CloseHandle(g_hFFmpegStdinWrite);
        CloseHandle(g_ffmpegPi.hProcess);
        CloseHandle(g_ffmpegPi.hThread);

        g_hFFmpegStdinWrite = NULL;
        ZeroMemory(&g_ffmpegPi, sizeof(g_ffmpegPi));
        g_isRecording = false;
    }
}

LRESULT CALLBACK OverlayProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
    case WM_LBUTTONDOWN:
        g_isDragging = true;
        g_ptStart.x = LOWORD(lParam);
        g_ptStart.y = HIWORD(lParam);
        g_selectedRect = { g_ptStart.x, g_ptStart.y, g_ptStart.x, g_ptStart.y };
        return 0;

    case WM_MOUSEMOVE:
        if (g_isDragging) {
            int curX = (short)LOWORD(lParam);
            int curY = (short)HIWORD(lParam);
            g_selectedRect.left = min(g_ptStart.x, curX);
            g_selectedRect.top = min(g_ptStart.y, curY);
            g_selectedRect.right = max(g_ptStart.x, curX);
            g_selectedRect.bottom = max(g_ptStart.y, curY);
            InvalidateRect(hWnd, NULL, TRUE);
        }
        return 0;

    case WM_LBUTTONUP:
        if (g_isDragging) {
            g_isDragging = false;
            g_selectionConfirmed = true;
            DestroyWindow(hWnd);
        }
        return 0;

    case WM_KEYDOWN:
        if (wParam == VK_ESCAPE) {
            g_selectionConfirmed = false;
            DestroyWindow(hWnd);
        }
        return 0;

    case WM_PAINT: {
```

```
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        if (g_isDragging) {
            HPEN hPen = CreatePen(PS_SOLID, 2, RGB(255, 0, 0));
            HGDIOBJ oldPen = SelectObject(hdc, hPen);
            HGDIOBJ oldBrush = SelectObject(hdc, GetStockObject(NULL_BRUSH));

            Rectangle(hdc, g_selectedRect.left, g_selectedRect.top, g_selectedRect.right,
g_selectedRect.bottom);

            SelectObject(hdc, oldPen);
            SelectObject(hdc, oldBrush);
            DeleteObject(hPen);
        }
        EndPaint(hWnd, &ps);
        return 0;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

void TriggerOverlay(HINSTANCE hInstance) {
    const wchar_t CLASS_NAME[] = L"MiniSnippingOverlay";

    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = OverlayProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hCursor = LoadCursor(NULL, IDC_CROSS);
    wc.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
    RegisterClassW(&wc);

    int screenW = GetSystemMetrics(SM_CXSCREEN);
    int screenH = GetSystemMetrics(SM_CYSCREEN);

    // Cửa sổ phủ mờ 25% (Alpha = 64)
    HWND hWndOverlay = CreateWindowExW(
        WS_EX_TOPMOST | WS_EX_LAYERED | WS_EX_TOOLWINDOW,
        CLASS_NAME, L"", WS_POPUP,
        0, 0, screenW, screenH,
        NULL, NULL, hInstance, NULL
    );

    SetLayeredWindowAttributes(hWndOverlay, 0, 64, LWA_ALPHA);
    ShowWindow(hWndOverlay, SW_SHOW);
    UpdateWindow(hWndOverlay);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    UnregisterClassW(CLASS_NAME, hInstance);

    if (g_selectionConfirmed) {
        int w = g_selectedRect.right - g_selectedRect.left;
        int h = g_selectedRect.bottom - g_selectedRect.top;
```

```
        if (w > 10 && h > 10) {
            StartRecording(g_selectedRect.left, g_selectedRect.top, w, h);
        }
    }
}

// Background Message Loop Window
LRESULT CALLBACK HiddenWndProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    if (uMsg == WM_HOTKEY && wParam == HOTKEY_ID) {
        if (g_isRecording) {
            StopRecording();
        } else {
            HINSTANCE hInst = (HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE);
            TriggerOverlay(hInst);
        }
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int
nCmdShow) {
    const wchar_t BG_CLASS[] = L"MiniRecorderBgClass";
    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = HiddenWndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = BG_CLASS;
    RegisterClassW(&wc);

    HWND hBgWnd = CreateWindowExW(0, BG_CLASS, L"", 0, 0, 0, 0, 0, HWND_MESSAGE, NULL,
hInstance, NULL);

    // Đăng ký Shift + PrintScreen
    if (!RegisterHotKey(hBgWnd, HOTKEY_ID, MOD_SHIFT, VK_SNAPSHOT)) {
        MessageBoxW(NULL, L"Không thể gán phím tắt Shift + PrintScreen!", L"Lỗi",
MB_ICONERROR);
        return 1;
    }

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    UnregisterHotKey(hBgWnd, HOTKEY_ID);
    return 0;
}
```

Cách biên dịch ra file .exe (< 100 KB)

Bạn có thể dùng MSVC (qua Developer Command Prompt) hoặc GCC (MinGW-w64):

• Nếu dùng GCC / MinGW:

```bash
g++ -O3 -mwindows main.cpp -o MiniRecorder.exe -lgdi32 -luser32
```

• Nếu dùng MSVC (cl.exe):

```dos
cl /O2 /Fe:MiniRecorder.exe main.cpp /link /SUBSYSTEM:WINDOWS user32.lib gdi32.lib
```

File MiniRecorder.exe tạo ra chỉ cần nằm cùng thư mục với ffmpeg.exe. Khi chạy, app sẽ ẩn hoàn toàn vào background và kích hoạt ngay khi bạn nhấn Shift + Print Screen.

YOU

PrtScr: chọn region chụp màn (copy clipboard)

Shift + prtscr: chọn region quay màn

Screenshot-yyyymmdd-hhmmdd-mss.png video nên tiền tố nào, lưu vào pictures/Screenshot

Videos thì sao?

GEMINI

Tiền tố chuẩn cho video

Tiền tố đối xứng và tự nhiên nhất với Screenshot- là:

```plaintext
Screenrecording-yyyyMMdd-HHmmss-fff.mp4
```

(hoặc dạng ngắn gọn, sạch sẽ hơn: Record-yyyyMMdd-HHmmss-fff.mp4 / Recording-yyyyMMdd-HHmmss-fff.mp4)

Nếu muốn đồng bộ tuyệt đối về mặt quy ước đặt tên:

• Ảnh chụp: Screenshot-20261109-091530-123.png
• Video quay: Screenrecording-20261109-091530-123.mp4

Lưu ý định dạng thời gian: Định dạng giây chuẩn là ss (seconds), còn fff hoặc mss đại diện cho mili-giây (milliseconds), tránh nhầm với dd (ngày).

Thư mục lưu chuẩn trên Windows cho Video

Windows có thư mục tiêu chuẩn chuyên dành cho quay màn hình (được Game Bar, Snipping Tool và Windows Media Foundation sử dụng):

```plaintext
%USERPROFILE%\Videos\Captures
```

(Đường dẫn thực tế: C:\Users\<TênUser>\Videos\Captures)

Lựa chọn nơi lưu:

1. Lưu vào Videos\Captures (Khuyên dùng):
   • Tách biệt hoàn toàn ảnh tĩnh (Pictures\Screenshots) và video nặng (Videos\Captures).
   • Tương thích với quy chuẩn mặc định của Windows Explorer, Windows Photos và Clipchamp.
2. Lưu vào Pictures\Screenshots chung với ảnh:
   • Chỉ nên dùng nếu bạn muốn gom toàn bộ dữ liệu vừa chụp/quay vào một nơi duy nhất để tiện mở ra chọn hoặc gửi nhanh mà không phải chuyển qua lại giữa 2 thư mục.

Cách lấy đường dẫn Known Folder chuẩn trong C++ Win32

Để không bị lỗi khi người dùng đổi tên tài khoản hoặc di chuyển thư mục Documents/Videos/Pictures sang ổ đĩa khác (D:, E:), hãy dùng API SHGetKnownFolderPath:

```c++
#include <shlobj.h>
#include <knownfolders.h>

// Lấy thư mục Pictures\Screenshots (cho ảnh)
PWSTR pszPathPictures = NULL;
if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Screenshots, 0, NULL, &pszPathPictures))) {
    // pszPathPictures = C:\Users\<User>\Pictures\Screenshots
    CoTaskMemFree(pszPathPictures);
}

// Lấy thư mục Videos\Captures (cho video)
PWSTR pszPathCaptures = NULL;
if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Captures, 0, NULL, &pszPathCaptures))) {
    // pszPathCaptures = C:\Users\<User>\Videos\Captures
    CoTaskMemFree(pszPathCaptures);
}
```

Bạn có muốn tích hợp luôn tính năng PrintScreen chụp ảnh copy clipboard vào code C++ không?

YOU

C++ có ui nhỏ tích chọn autostart ko

GEMINI

Có. Trong Win32 C++, cách chuẩn và gọn nhất để có UI nhỏ bật/tắt Auto-start là tạo System Tray Icon (khay hệ thống) ở góc dưới bên phải: khi chuột phải vào icon sẽ hiện menu ngữ cảnh có ô tích [] Start with Windows và nút Exit.

Cơ chế này ghi trực tiếp giá trị vào registry HKCU\Software\Microsoft\Windows\CurrentVersion\Run, không cần mở cửa sổ rườm rà và không cần quyền Administrator.

Cách hoạt động

1. Kiểm tra trạng thái: Đọc Registry xem khóa MiniScreenRecorder đã tồn tại chưa.
2. Menu khay hệ thống:
   - Chuột phải vào Tray Icon $\rightarrow$ menu hiện lên.
   - Nếu đã bật Auto-start: mục menu có dấu tick .
   - Click vào mục đó: đảo trạng thái (thêm hoặc xóa key trong Registry) và cập nhật lại dấu tick.
   - Click Exit: gỡ icon, hủy hotkey và thoát chương trình.

Mã nguồn mẫu Win32 C++ tích hợp Tray Menu & Auto-start

Bạn có thể gắn đoạn code này trực tiếp vào file C++ của tool:

```c++
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <shellapi.h>
#include <string>
```

```
#define WM_TRAYICON       (WM_USER + 1)
#define IDM_AUTOSTART     1001
#define IDM_EXIT          1002

const wchar_t REG_KEY[] = L"Software\\Microsoft\\Windows\\CurrentVersion\\Run";
const wchar_t APP_NAME[] = L"MiniScreenRecorder";

// Kiểm tra xem ứng dụng đã có trong Registry Run chưa
bool IsAutoStartEnabled() {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, REG_KEY, 0, KEY_READ, &hKey) == ERROR_SUCCESS) {
        DWORD type = 0;
        LONG res = RegQueryValueExW(hKey, APP_NAME, NULL, &type, NULL, NULL);
        RegCloseKey(hKey);
        return (res == ERROR_SUCCESS);
    }
    return false;
}

// Bật hoặc tắt Auto-start
void SetAutoStart(bool enable) {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, REG_KEY, 0, KEY_SET_VALUE, &hKey) == ERROR_SUCCESS)
{
        if (enable) {
            wchar_t path[MAX_PATH];
            GetModuleFileNameW(NULL, path, MAX_PATH);
            // Bọc dấu ngoặc kép để đường dẫn có khoảng trắng không bị lỗi
            std::wstring cmd = L"\"" + std::wstring(path) + L"\"";
            RegSetValueExW(hKey, APP_NAME, 0, REG_SZ,
                (const BYTE*)cmd.c_str(), (DWORD)((cmd.length() + 1) * sizeof(wchar_t)));
        } else {
            RegDeleteValueW(hKey, APP_NAME);
        }
        RegCloseKey(hKey);
    }
}

// Hiển thị menu ngữ cảnh tại vị trí con trỏ chuột
void ShowContextMenu(HWND hWnd) {
    POINT pt;
    GetCursorPos(&pt);

    HMENU hMenu = CreatePopupMenu();
    bool autoStart = IsAutoStartEnabled();

    // Thêm mục Start with Windows với cờ check/uncheck
    UINT checkFlag = autoStart ? MF_CHECKED : MF_UNCHECKED;
    AppendMenuW(hMenu, MF_STRING | checkFlag, IDM_AUTOSTART, L"Start with Windows");
    AppendMenuW(hMenu, MF_SEPARATOR, 0, NULL);
    AppendMenuW(hMenu, MF_STRING, IDM_EXIT, L"Exit");

    // Fix lỗi menu không tự ẩn khi bấm chuột ra ngoài
    SetForegroundWindow(hWnd);
    TrackPopupMenu(hMenu, TPM_BOTTOMALIGN | TPM_LEFTALIGN, pt.x, pt.y, 0, hWnd, NULL);
    DestroyMenu(hMenu);
}

// Xử lý thông điệp trong WndProc chính
LRESULT CALLBACK MainWndProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    static NOTIFYICONDATAW nid = { 0 };
```

```
    switch (uMsg) {
    case WM_CREATE: {
        // Đăng ký System Tray Icon
        nid.cbSize = sizeof(NOTIFYICONDATAW);
        nid.hWnd = hWnd;
        nid.uID = 1;
        nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
        nid.uCallbackMessage = WM_TRAYICON;
        nid.hIcon = LoadIcon(NULL, IDI_APPLICATION); // Dùng icon app mặc định
        wcscpy_s(nid.szTip, L"Mini Screen Recorder");
        Shell_NotifyIconW(NIM_ADD, &nid);
        return 0;
    }

    case WM_TRAYICON:
        // Nhấn chuột phải hoặc chuột trái vào Icon khay hệ thống
        if (lParam == WM_RBUTTONUP || lParam == WM_LBUTTONUP) {
            ShowContextMenu(hWnd);
        }
        return 0;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_AUTOSTART:
            SetAutoStart(!IsAutoStartEnabled());
            break;
        case IDM_EXIT:
            DestroyWindow(hWnd);
            break;
        }
        return 0;

    case WM_DESTROY:
        // Gỡ icon khỏi tray khi thoát
        Shell_NotifyIconW(NIM_DELETE, &nid);
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProcW(hWnd, uMsg, wParam, lParam);
}
```

Biên dịch

Khi biên dịch với GCC (MinGW) hoặc MSVC, bạn chỉ cần liên kết thêm thư viện shell32.lib và advapi32.lib (cho Registry):

• MinGW GCC:

```bash
  g++ -O3 -mwindows main.cpp -o MiniRecorder.exe -lgdi32 -luser32 -lshell32 -ladvapi32
```

• MSVC (cl.exe):

```dos
  cl /O2 /Fe:MiniRecorder.exe main.cpp /link /SUBSYSTEM:WINDOWS user32.lib gdi32.lib
shell32.lib advapi32.lib
```

File sau khi biên dịch vẫn giữ được dung lượng cực nhỏ (dưới 100 KB), chạy ẩn dưới góc phải màn hình và

quản lý tự khởi động cùng Windows chỉ với 1 cú click chuột.

YOU

Prefix video hơi rối

GEMINI

Nếu dùng Screenrecording- dài và rối mắt, bạn có thể rút gọn lại theo đúng 1 từ đơn để vừa ngắn, vừa tự nhiên và giữ trật tự sắp xếp (sort alphabet) gọn gàng trong thư mục.

Một số cách đặt prefix video tinh giản, dễ nhìn:

1. Cặp từ đối xứng, ngắn nhất (Khuyên dùng)

• Ảnh: Shot-20261109-091530.png
• Video: Clip-20261109-091530.mp4

Ưu điểm: Từ Clip và Shot đều chỉ có 4 ký tự, nhìn vào biết ngay đâu là video, đâu là ảnh chụp nhanh, độ dài tên file đồng đều tăm tắp.

2. Dùng từ chuẩn chức năng (3 ký tự)

• Ảnh: Cap-20261109-091530.png (Capture)
• Video: Rec-20261109-091530.mp4 (Record)

Ưu điểm: Cực gọn, tiết kiệm không gian hiển thị trên Windows Explorer khi để chế độ xem danh sách (Details view).

3. Giữ nguyên Screenshot- cho ảnh

Nếu ảnh vẫn giữ chuẩn Screenshot-, thì video chỉ cần:

• Ảnh: Screenshot-20261109-091530.png
• Video: Screenrec-20261109-091530.mp4 (hoặc Video-20261109-091530.mp4)

Mẹo: Bỏ phần mili-giây (-mss) nếu không cần thiết

Chuỗi -123 mili-giây ở đuôi thường là nguyên nhân chính khiến tên file trông rối rắm. Với việc quay video hoặc chụp màn hình thủ công bằng tay, chỉ cần tới giây (yyyyMMdd-HHmmss) là đủ đảm bảo không bao giờ bị trùng tên, tên file sẽ thanh thoát hơn rất nhiều:

• Rối: Screenrecording-20261109-091530-842.mp4
• Gọn: Clip-20261109-091530.mp4

Bạn muốn chọn cặp nào (Clip/Shot hay Rec/Cap) để mình cập nhật lại code định dạng chuỗi?

YOU

Screen-recording đi nhỉ

GEMINI

Tiền tố Screen-recording- (hoặc Screen-recording_) rất hợp lý vì:

• Tách từ rõ ràng: Dấu gạch nối giúp mắt phân biệt ngay giữa "Screen" và "Recording", không bị dính chùm như Screenrecording.
• Cân xứng với ảnh:
  • Ảnh: Screenshot-20261109-091530.png
  • Video: Screen-recording-20261109-091530.mp4
• Dễ lọc (Filter/Search): Khi gõ Screen- trong thanh tìm kiếm Windows Explorer, cả ảnh chụp lẫn video quay màn hình đều sẽ xuất hiện cùng nhau theo đúng thứ tự thời gian.

Cập nhật checklist dự án

Yêu cầu kiểm thử & Hoàn thiện Mini Screen Recorder

Đoạn code C++ tạo tên file chuẩn (đã bỏ mili-giây)

```c++
// Tạo chuỗi timestamp theo định dạng Screen-recording-yyyyMMdd-HHmmss.mp4
time_t rawtime;
struct tm timeinfo;
char filename[128];
time(&rawtime);
localtime_s(&timeinfo, &rawtime);

// Format: Screen-recording-20261109-091600.mp4
strftime(filename, sizeof(filename), "Screen-recording-%Y%m%d-%H%M%S.mp4", &timeinfo);

std::string fullPath = std::string(outputDir) + "\\" + filename;
```

YOU

Nên embed vào exe ffmpeg ko

GEMINI

Không nên nhúng trực tiếp ffmpeg.exe vào file .exe chính.

Cách tối ưu nhất trên Windows là để ffmpeg.exe nằm bên cạnh file .exe chính (hoặc tự động tải về thư mục %LOCALAPPDATA% ở lần chạy đầu tiên giống cách ShareX làm).

So sánh thực tế: Để ngoài vs. Nhúng vào trong (Embed)

Tiêu chí	Để ffmpeg.exe bên cạnh (Khuyên dùng)	Nhúng vào trong Resource .exe
Dung lượng file .exe	Cực nhẹ (< 100 KB)	Phình to (50 MB – 80 MB)
Tốc độ khởi động	Gọi process thực thi ngay lập tức	Phải ghi byte từ RAM ra ổ cứng (Temp) trước
Ghi đĩa & Ổ cứng	Không tốn I/O rác	Ghi và xóa file liên tục ở thư mục %TEMP%
Báo động Antivirus	Ít bị nghi ngờ	Rất dễ bị Windows Defender / AV chặn nhầm vì hành vi "dropper"
Cập nhật FFmpeg	Thay file mới bất cứ lúc nào	Phải compile lại toàn bộ mã nguồn

3 lý do kỹ thuật không nên Embed

1. Windows không thể chạy trực tiếp file thực thi từ RAM:
  Windows API (CreateProcess) chỉ khởi chạy được tiến trình từ một đường dẫn file thực tế trên ổ đĩa. Do đó, nếu nhúng ffmpeg.exe dưới dạng Resource/Bytes bên trong file C++, mỗi lần chạy app buộc phải trích xuất (dump) file đó ra thư mục tạm (%TEMP%\ffmpeg.exe), vừa tốn thời gian vừa ghi đĩa thừa thãi.
2. Dễ bị Windows Defender gắn cờ mã độc (False Positive):

Hành vi "một file .exe giải nén một file .exe khác vào thư mục tạm rồi âm thầm thực thi không hiện cửa sổ" là đặc điểm nhận diện kinh điển của Trojan Dropper. Các phần mềm diệt virus rất dễ quét và chặn ứng dụng của bạn.

3. Vấn đề bản quyền GPL/LGPL:

FFmpeg đi kèm giấy phép LGPL/GPL. Khi để ffmpeg.exe thành một file rời độc lập và gọi qua lệnh CLI, dự án của bạn hoàn toàn hợp lệ về mặt bản quyền mà không bị ràng buộc phải mở toàn bộ mã nguồn theo giấy phép copyleft.

Giải pháp chuyên nghiệp nếu muốn phân phối gọn gàng

Nếu bạn muốn khi chia sẻ cho người khác chỉ có đúng 1 file duy nhất, hãy áp dụng cơ chế của ShareX:

• File .exe chính vẫn giữ nguyên dung lượng siêu nhẹ (~100 KB).
• Khi người dùng nhấn quay lần đầu tiên: kiểm tra nếu chưa thấy ffmpeg.exe trong thư mục hoặc trong %LOCALAPPDATA%\MiniRecorder, app sẽ hiện thông báo 1-click tải bản tĩnh tối giản về rồi lưu cố định tại đó.

Bạn có muốn xem đoạn code C++ kiểm tra và tự tải ffmpeg.exe về %LOCALAPPDATA% không?

YOU

Tên opensource nên là gì?

GEMINI

Một cái tên mã nguồn mở (open-source) hay cho dự án dạng này cần đáp ứng 3 tiêu chí: ngắn gọn (dễ gõ lệnh), mô tả đúng bản chất (siêu nhẹ / snapshot / record), và chưa bị trùng quá nhiều trên GitHub.

Dưới đây là các hướng đặt tên phù hợp nhất tùy theo phong cách bạn muốn:

1. Phong cách Unix / Hacker / CLI (Ngắn nhất, 4–5 ký tự)

Những cái tên kiểu này rất được cộng đồng developer ưa chuộng vì gõ Win + R hoặc mở terminal gọi tên cực nhanh:

• snpr (viết tắt của Snipper / Snapper): Nghe sắc sảo, dứt khoát, gợi liên tưởng đến việc kéo chọn vùng chụp/quay nhanh.
• recx (Record + X): Tinh thần kế thừa từ ShareX nhưng tập trung thuần vào quay/chụp.
• minirec (Mini Recorder): Rõ ràng, bình dân, ai đọc cũng hiểu ngay tính năng mà không cần giải thích.
• cropr (Cropper): Nhấn mạnh vào hành động kéo crop vùng màn hình để ghi lại.

2. Phong cách Tiện ích Nhẹ & Tinh giản (Minimalist & Modern)

Nhấn mạnh vào kích thước file tí hon và tốc độ phản hồi tức thì:

• MicroCap (Micro Capture): Gợi cảm giác siêu nhẹ, chỉ vài chục KB, ăn đứt các tool cồng kềnh hàng trăm MB.
• SnapClip: Ghép giữa Snap (chụp/chọn nhanh) và Clip (đoạn video ngắn).
• TinyRec hoặc NanoRec: Trực quan, định vị ngay là phiên bản recorder nhỏ nhất có thể.
• ShotClip: Tương ứng hoàn hảo với 2 chức năng bạn làm: Chụp ảnh (Shot) và Quay màn (Clip).

3. Phong cách chơi chữ với Phím tắt

Lấy cảm hứng từ chính phím kích hoạt cốt lõi của công cụ:

• PrtScx (PrintScreen Extended): Biến phím PrtScn mặc định thành công cụ đa năng (chụp + quay).
• ShiftPrt: Đặt theo đúng tổ hợp phím kích hoạt chức năng quay màn hình (Shift + PrtScn).

Gợi ý phù hợp nhất: MicroCap hoặc ShotClip

• Nếu muốn đặt tên repo GitHub & file thực thi:
  • Repo: github.com/username/ShotClip (https://github.com/username/ShotClip) (hoặc MicroCap)
  • File thực thi: shotclip.exe (hoặc minirec.exe)
• Dòng mô tả (Tagline) gợi ý cho README:
  > "A featherweight (<100KB), zero-bloat Windows region screenshot & screen recording tool powered by Win32 & FFmpeg."

YOU

MiniCapture thì sao

GEMINI

MiniCapture là một cái tên rất chuẩn, an toàn và chuyên nghiệp cho dự án dạng này.

Đánh giá tên MiniCapture

• Mô tả trực diện bản chất: Người dùng nhìn vào là hiểu ngay: "Mini" (siêu nhẹ, không rườm rà) + "Capture" (chụp/quay giữ lại màn hình).
• Độ bao quát cao: Từ "Capture" trong tiếng Anh bao hàm cả Screen Capture (chụp ảnh tĩnh) và Video Capture (ghi hình video). Điều này giúp app không bị gò bó nếu sau này bạn mở rộng thêm tính năng quay âm thanh hay copy link.
• Tên file binary đẹp: minicapture.exe gõ trên Windows Run (Win + R) hoặc command line rất xuôi tay và tự nhiên.

Điểm cần lưu ý về mức độ phổ biến

Từ "Mini" kết hợp với "Capture" khá thông dụng nên trên GitHub sẽ có một số repo sinh viên hoặc demo nhỏ trùng tên này. Tuy nhiên, điều này không thành vấn đề nếu bạn tạo một tagline sắc nét để định vị ngay sự khác biệt:

MiniCapture

A featherweight (<100KB) native Windows screen recorder & region snipper powered by Win32 API and FFmpeg.

Tùy biến biến thể (nếu muốn độc lạ hơn)

Nếu muốn tên repo và tên lệnh ngắn hơn 1 nhịp:

• MiniCap: Gọn hơn 4 ký tự, đọc rất lướt (minicap.exe).
• MiniCapture-win: Phân biệt rõ là bản native chuyên cho Windows.

Nếu bạn thích sự chỉn chu và rõ ràng, chốt MiniCapture là hoàn toàn hợp lý.

YOU

Mincap

GEMINI

Mincap là một lựa chọn xuất sắc, thậm chí còn bén và cá tính hơn MiniCapture:

• Cực ngắn (6 ký tự): Gõ phím mincap trong terminal hay hộp thoại Run (Win + R) chỉ mất chưa tới 0.5 giây.
• Đúng phong cách Unix/C++ utilities: Những tool kinh điển thường tối giản tên gọi theo dạng này (như libpcap, xclip, scrcpy, ffmpeg).
• Định danh file build sạch sẽ: mincap.exe tạo cảm giác nhẹ tênh, gọn gàng và không thừa thãi bất kỳ ký tự nào.
• Ít đụng hàng: Khả năng trùng repo hoặc nhầm lẫn thương hiệu thấp hơn nhiều so với các tên chung chung như "MiniCapture".

Gợi ý hoàn thiện bộ nhận diện cho Mincap

• Tên dự án / Binary: Mincap / mincap.exe
• Short Description: A minimalist, ultra-lightweight (<100KB) native Windows region snipper & screen recorder powered by Win32 & FFmpeg.
• Phím tắt cốt lõi:
  • PrintScreen: Kéo chọn vùng $\rightarrow$ Lưu file ảnh + Copy Clipboard.
  • Shift + PrintScreen: Kéo chọn vùng $\rightarrow$ Quay MP4 $\rightarrow$ Bấm lại để lưu.

Checklist hoàn thiện dự án Mincap

YOU

Dsnap rồi prfix dnap-screenshot-yyyymmdd-hhmmss.png và dsnap-recording-yyyymmdd-hhmmss.mp4

GEMINI

Cái tên dsnap rất ngắn gọn, độc đáo và mang đậm dấu ấn riêng. Quy chuẩn đặt tên file theo cặp này cực kỳ chỉn chu:

• Ảnh chụp: dsnap-screenshot-yyyymmdd-hhmmss.png
• Video quay: dsnap-recording-yyyymmdd-hhmmss.mp4

Dưới đây là toàn bộ mã nguồn C++ hoàn chỉnh gom gọn trong 1 file duy nhất (dsnap.cpp), tích hợp đầy đủ:

1. PrintScreen: Kéo chọn vùng $\rightarrow$ Lưu file ảnh PNG vào Pictures\Screenshots + Tự copy bitmap vào Clipboard để dán ngay.
2. Shift + PrintScreen: Kéo chọn vùng $\rightarrow$ Bật FFmpeg quay MP4 lưu vào Videos\Captures $\rightarrow$ Bấm lại lần nữa để dừng và lưu an toàn.
3. Tray Icon + Auto-start: Chạy ngầm dưới khay hệ thống, chuột phải có tùy chọn Start with Windows và Exit.

Mã nguồn dsnap.cpp

```c++
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <shellapi.h>
#include <shlobj.h>
#include <knownfolders.h>
#include <gdiplus.h>
#include <string>
#include <ctime>
#include <vector>
```

```
#pragma comment(lib, "gdiplus.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "advapi32.lib")

using namespace Gdiplus;

#define HOTKEY_PHOTO   9001
#define HOTKEY_VIDEO   9002
#define WM_TRAYICON    (WM_USER + 1)
#define IDM_AUTOSTART  1001
#define IDM_EXIT       1002

const wchar_t APP_NAME[] = L"dsnap";
const wchar_t REG_KEY[]  = L"Software\\Microsoft\\Windows\\CurrentVersion\\Run";

// Trạng thái Recorder
HANDLE g_hFFmpegStdinWrite = NULL;
PROCESS_INFORMATION g_ffmpegPi = { 0 };
bool g_isRecording = false;

// Trạng thái Overlay Snipping
enum Mode { MODE_NONE, MODE_PHOTO, MODE_VIDEO };
Mode g_currentMode = MODE_NONE;
bool g_isDragging = false;
POINT g_ptStart = { 0, 0 };
RECT g_selectedRect = { 0, 0, 0, 0 };
bool g_confirmed = false;

// Auto-start Registry
bool IsAutoStartEnabled() {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, REG_KEY, 0, KEY_READ, &hKey) == ERROR_SUCCESS) {
        DWORD type = 0;
        LONG res = RegQueryValueExW(hKey, APP_NAME, NULL, &type, NULL, NULL);
        RegCloseKey(hKey);
        return (res == ERROR_SUCCESS);
    }
    return false;
}

void SetAutoStart(bool enable) {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, REG_KEY, 0, KEY_SET_VALUE, &hKey) == ERROR_SUCCESS)
{
        if (enable) {
            wchar_t path[MAX_PATH];
            GetModuleFileNameW(NULL, path, MAX_PATH);
            std::wstring cmd = L"\"" + std::wstring(path) + L"\"";
            RegSetValueExW(hKey, APP_NAME, 0, REG_SZ, (const BYTE*)cmd.c_str(),
(DWORD)((cmd.length() + 1) * sizeof(wchar_t)));
        } else {
            RegDeleteValueW(hKey, APP_NAME);
        }
        RegCloseKey(hKey);
    }
}

// Lấy CLSID cho Encoder PNG trong GDI+
```

```
int GetEncoderClsid(const WCHAR* format, CLSID* pClsid) {
    UINT num = 0, size = 0;
    GetImageEncodersSize(&num, &size);
    if (size == 0) return -1;

    std::vector<BYTE> buffer(size);
    ImageCodecInfo* pImageCodecInfo = (ImageCodecInfo*)buffer.data();
    GetImageEncoders(num, size, pImageCodecInfo);

    for (UINT j = 0; j < num; ++j) {
        if (wcscmp(pImageCodecInfo[j].MimeType, format) == 0) {
            *pClsid = pImageCodecInfo[j].Clsid;
            return j;
        }
    }
    return -1;
}

// Tạo chuỗi thời gian yyyymmdd-hhmmss
std::string GetTimestamp() {
    time_t rawtime;
    struct tm ti;
    char buf[64];
    time(&rawtime);
    localtime_s(&ti, &rawtime);
    strftime(buf, sizeof(buf), "%Y%m%d-%H%M%S", &ti);
    return std::string(buf);
}

// Xử lý Chụp ảnh (Lưu PNG + Copy Clipboard)
void SaveScreenshot(int x, int y, int w, int h) {
    HDC hScreenDC = GetDC(NULL);
    HDC hMemoryDC = CreateCompatibleDC(hScreenDC);
    HBITMAP hBitmap = CreateCompatibleBitmap(hScreenDC, w, h);
    HGDIOBJ hOldBitmap = SelectObject(hMemoryDC, hBitmap);

    BitBlt(hMemoryDC, 0, 0, w, h, hScreenDC, x, y, SRCCOPY);

    // 1. Copy bitmap vào Clipboard
    OpenClipboard(NULL);
    EmptyClipboard();
    SetClipboardData(CF_BITMAP, hBitmap);
    CloseClipboard();

    // 2. Lưu file PNG vào Pictures\Screenshots
    PWSTR pPicturesPath = NULL;
    std::wstring outDir = L".";
    if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Screenshots, 0, NULL, &pPicturesPath))) {
        outDir = pPicturesPath;
        CoTaskMemFree(pPicturesPath);
    }

    std::string ts = GetTimestamp();
    std::wstring filePath = outDir + L"\\dsnap-screenshot-" + std::wstring(ts.begin(),
ts.end()) + L".png";

    Bitmap* gdiBitmap = new Bitmap(hBitmap, NULL);
    CLSID pngClsid;
    if (GetEncoderClsid(L"image/png", &pngClsid) != -1) {
        gdiBitmap->Save(filePath.c_str(), &pngClsid, NULL);
    }
```

```
    delete gdiBitmap;
    SelectObject(hMemoryDC, hOldBitmap);
    DeleteDC(hMemoryDC);
    ReleaseDC(NULL, hScreenDC);
}

// Xử lý Quay Video bằng FFmpeg
void StartRecording(int x, int y, int w, int h) {
    w -= (w % 2);
    h -= (h % 2);
    if (w <= 0 || h <= 0) return;

    PWSTR pVideosPath = NULL;
    std::string outDir = ".";
    if (SUCCEEDED(SHGetKnownFolderPath(FOLDERID_Captures, 0, NULL, &pVideosPath))) {
        int len = WideCharToMultiByte(CP_UTF8, 0, pVideosPath, -1, NULL, 0, NULL, NULL);
        if (len > 0) {
            std::vector<char> buf(len);
            WideCharToMultiByte(CP_UTF8, 0, pVideosPath, -1, buf.data(), len, NULL, NULL);
            outDir = std::string(buf.data());
        }
        CoTaskMemFree(pVideosPath);
    }

    std::string outputPath = outDir + "\\dsnap-recording-" + GetTimestamp() + ".mp4";

    SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
    HANDLE hStdinRead = NULL;
    if (!CreatePipe(&hStdinRead, &g_hFFmpegStdinWrite, &sa, 0)) return;
    SetHandleInformation(g_hFFmpegStdinWrite, HANDLE_FLAG_INHERIT, 0);

    STARTUPINFOA si = { 0 };
    si.cb = sizeof(si);
    si.hStdInput = hStdinRead;
    si.dwFlags |= STARTF_USESTDHANDLES;

    std::string cmd = "ffmpeg.exe -y -f gdigrab -framerate 30 -offset_x " +
std::to_string(x) +
        " -offset_y " + std::to_string(y) + " -video_size " + std::to_string(w) + "x" +
std::to_string(h) +
        " -i desktop -c:v libx264 -preset ultrafast -crf 23 -pix_fmt yuv420p \"" +
outputPath + "\"";

    BOOL success = CreateProcessA(NULL, &cmd[0], NULL, NULL, TRUE, CREATE_NO_WINDOW, NULL,
NULL, &si, &g_ffmpegPi);
    CloseHandle(hStdinRead);

    if (success) {
        g_isRecording = true;
    } else {
        CloseHandle(g_hFFmpegStdinWrite);
        g_hFFmpegStdinWrite = NULL;
    }
}

void StopRecording() {
    if (g_isRecording && g_hFFmpegStdinWrite != NULL) {
        DWORD bytesWritten;
        const char* q = "q\n";
        WriteFile(g_hFFmpegStdinWrite, q, (DWORD)strlen(q), &bytesWritten, NULL);
```

```
        WaitForSingleObject(g_ffmpegPi.hProcess, 3000);

        CloseHandle(g_hFFmpegStdinWrite);
        CloseHandle(g_ffmpegPi.hProcess);
        CloseHandle(g_ffmpegPi.hThread);

        g_hFFmpegStdinWrite = NULL;
        ZeroMemory(&g_ffmpegPi, sizeof(g_ffmpegPi));
        g_isRecording = false;
    }
}

// Overlay Window Proc
LRESULT CALLBACK OverlayProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
    case WM_LBUTTONDOWN:
        g_isDragging = true;
        g_ptStart.x = LOWORD(lParam);
        g_ptStart.y = HIWORD(lParam);
        g_selectedRect = { g_ptStart.x, g_ptStart.y, g_ptStart.x, g_ptStart.y };
        return 0;

    case WM_MOUSEMOVE:
        if (g_isDragging) {
            int curX = (short)LOWORD(lParam);
            int curY = (short)HIWORD(lParam);
            g_selectedRect.left = min(g_ptStart.x, curX);
            g_selectedRect.top = min(g_ptStart.y, curY);
            g_selectedRect.right = max(g_ptStart.x, curX);
            g_selectedRect.bottom = max(g_ptStart.y, curY);
            InvalidateRect(hWnd, NULL, TRUE);
        }
        return 0;

    case WM_LBUTTONUP:
        if (g_isDragging) {
            g_isDragging = false;
            g_confirmed = true;
            DestroyWindow(hWnd);
        }
        return 0;

    case WM_KEYDOWN:
        if (wParam == VK_ESCAPE) {
            g_confirmed = false;
            DestroyWindow(hWnd);
        }
        return 0;

    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hWnd, &ps);
        if (g_isDragging) {
            COLORREF borderColor = (g_currentMode == MODE_PHOTO) ? RGB(0, 162, 232) :
RGB(255, 50, 50);
            HPEN hPen = CreatePen(PS_SOLID, 2, borderColor);
            HGDIOBJ oldPen = SelectObject(hdc, hPen);
            HGDIOBJ oldBrush = SelectObject(hdc, GetStockObject(NULL_BRUSH));

            Rectangle(hdc, g_selectedRect.left, g_selectedRect.top, g_selectedRect.right,
g_selectedRect.bottom);
```

```
            SelectObject(hdc, oldPen);
            SelectObject(hdc, oldBrush);
            DeleteObject(hPen);
        }
        EndPaint(hWnd, &ps);
        return 0;
    }
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

void TriggerOverlay(HINSTANCE hInstance, Mode mode) {
    g_currentMode = mode;
    g_confirmed = false;

    const wchar_t CLASS_NAME[] = L"DsnapOverlay";
    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = OverlayProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hCursor = LoadCursor(NULL, IDC_CROSS);
    wc.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
    RegisterClassW(&wc);

    int screenW = GetSystemMetrics(SM_CXSCREEN);
    int screenH = GetSystemMetrics(SM_CYSCREEN);

    HWND hWnd = CreateWindowExW(WS_EX_TOPMOST | WS_EX_LAYERED | WS_EX_TOOLWINDOW,
        CLASS_NAME, L"", WS_POPUP, 0, 0, screenW, screenH, NULL, NULL, hInstance, NULL);

    SetLayeredWindowAttributes(hWnd, 0, 60, LWA_ALPHA);
    ShowWindow(hWnd, SW_SHOW);
    UpdateWindow(hWnd);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    UnregisterClassW(CLASS_NAME, hInstance);

    if (g_confirmed) {
        int w = g_selectedRect.right - g_selectedRect.left;
        int h = g_selectedRect.bottom - g_selectedRect.top;
        if (w > 10 && h > 10) {
            if (mode == MODE_PHOTO) SaveScreenshot(g_selectedRect.left, g_selectedRect.top,
w, h);
            else if (mode == MODE_VIDEO) StartRecording(g_selectedRect.left,
g_selectedRect.top, w, h);
        }
    }
}

// Background Window Proc & Tray
LRESULT CALLBACK WndProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    static NOTIFYICONDATAW nid = { 0 };
```

```
    switch (uMsg) {
    case WM_CREATE:
        nid.cbSize = sizeof(NOTIFYICONDATAW);
        nid.hWnd = hWnd;
        nid.uID = 1;
        nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
        nid.uCallbackMessage = WM_TRAYICON;
        nid.hIcon = LoadIcon(NULL, IDI_APPLICATION);
        wcscpy_s(nid.szTip, L"dsnap");
        Shell_NotifyIconW(NIM_ADD, &nid);
        return 0;

    case WM_TRAYICON:
        if (lParam == WM_RBUTTONUP || lParam == WM_LBUTTONUP) {
            POINT pt;
            GetCursorPos(&pt);
            HMENU hMenu = CreatePopupMenu();
            UINT flag = IsAutoStartEnabled() ? MF_CHECKED : MF_UNCHECKED;
            AppendMenuW(hMenu, MF_STRING | flag, IDM_AUTOSTART, L"Start with Windows");
            AppendMenuW(hMenu, MF_SEPARATOR, 0, NULL);
            AppendMenuW(hMenu, MF_STRING, IDM_EXIT, L"Exit");
            SetForegroundWindow(hWnd);
            TrackPopupMenu(hMenu, TPM_BOTTOMALIGN | TPM_LEFTALIGN, pt.x, pt.y, 0, hWnd,
NULL);
            DestroyMenu(hMenu);
        }
        return 0;

    case WM_COMMAND:
        if (LOWORD(wParam) == IDM_AUTOSTART) SetAutoStart(!IsAutoStartEnabled());
        else if (LOWORD(wParam) == IDM_EXIT) DestroyWindow(hWnd);
        return 0;

    case WM_HOTKEY:
        if (wParam == HOTKEY_PHOTO) {
            TriggerOverlay((HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE), MODE_PHOTO);
        } else if (wParam == HOTKEY_VIDEO) {
            if (g_isRecording) StopRecording();
            else TriggerOverlay((HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE),
MODE_VIDEO);
        }
        return 0;

    case WM_DESTROY:
        if (g_isRecording) StopRecording();
        Shell_NotifyIconW(NIM_DELETE, &nid);
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int) {
    ULONG_PTR gdiplusToken;
    GdiplusStartupInput gdiplusStartupInput;
    GdiplusStartup(&gdiplusToken, &gdiplusStartupInput, NULL);

    const wchar_t BG_CLASS[] = L"DsnapBackground";
    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
```

```
    wc.lpszClassName = BG_CLASS;
    RegisterClassW(&wc);

    HWND hWnd = CreateWindowExW(0, BG_CLASS, L"", 0, 0, 0, 0, 0, HWND_MESSAGE, NULL,
hInstance, NULL);

    // Gán phím: PrintScreen (Chụp) & Shift + PrintScreen (Quay)
    RegisterHotKey(hWnd, HOTKEY_PHOTO, 0, VK_SNAPSHOT);
    RegisterHotKey(hWnd, HOTKEY_VIDEO, MOD_SHIFT, VK_SNAPSHOT);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    UnregisterHotKey(hWnd, HOTKEY_PHOTO);
    UnregisterHotKey(hWnd, HOTKEY_VIDEO);
    GdiplusShutdown(gdiplusToken);
    return 0;
}
```

Cập nhật checklist dự án

Checklist hoàn thiện dự án dsnap

Lệnh biên dịch ra dsnap.exe (< 100 KB)

• Dùng MinGW GCC:

```bash
g++ -O3 -mwindows dsnap.cpp -o dsnap.exe -lgdi32 -luser32 -lshell32 -ladvapi32 -lgdiplus
```

• Dùng MSVC (Developer Command Prompt):

```dos
cl /O2 /Fe:dsnap.exe dsnap.cpp /link /SUBSYSTEM:WINDOWS
```

YOU

Làm sao override những app cod sẵn và truoec đó phím tắt này?

GEMINI

Để chiếm quyền (override) phím tắt PrintScreen và Shift + PrintScreen khi các app khác (hoặc chính Windows Snipping Tool / OneDrive) đã đăng ký trước, bạn không thể dùng RegisterHotKey.

Quy tắc của Windows với RegisterHotKey: Ai đăng ký trước thì chiếm, app chạy sau gọi RegisterHotKey sẽ trả về FALSE (thất bại).

Giải pháp chuẩn kỹ thuật và triệt để nhất là dùng Low-Level Keyboard Hook (SetWindowsHookEx với WH_KEYBOARD_LL).

Cơ chế của Low-Level Keyboard Hook

1. Ưu tiên cao nhất trong chuỗi nhập liệu: Hook cấp thấp đứng trước cả hàng đợi thông điệp (Message

Queue) và bộ lọc của RegisterHotKey.
2. Nuốt phím (Consume Key): Khi phát hiện phím VK_SNAPSHOT (PrintScreen), hàm callback của bạn trả về 1 thay vì gọi CallNextHookEx. Thao tác này "triệt tiêu" tín hiệu phím ngay tại chỗ — các app đăng ký trước đó (kể cả Windows Snipping Tool) sẽ hoàn toàn không nhận được phím này nữa.

Code mẫu tích hợp Hook override phím tắt

Thay thế toàn bộ đoạn gọi RegisterHotKey / WM_HOTKEY bằng đoạn mã Hook dưới đây:

```c++
#define WIN32_LEAN_AND_MEAN
#include <windows.h>

HHOOK g_hKeyboardHook = NULL;
HWND g_hMainWnd = NULL;

#define WM_DSNAP_TRIGGER (WM_USER + 100)
#define TRIGGER_PHOTO    1
#define TRIGGER_VIDEO    2

// Hàm Hook chạy ở tầng thấp nhất của hệ điều hành
LRESULT CALLBACK LowLevelKeyboardProc(int nCode, WPARAM wParam, LPARAM lParam) {
    if (nCode == HC_ACTION) {
        KBDLLHOOKSTRUCT* pKey = (KBDLLHOOKSTRUCT*)lParam;

        // Chỉ bắt khi phím được nhấn xuống (WM_KEYDOWN hoặc WM_SYSKEYDOWN)
        if (wParam == WM_KEYDOWN || wParam == WM_SYSKEYDOWN) {
            if (pKey->vkCode == VK_SNAPSHOT) {
                // Kiểm tra xem phím SHIFT có đang được giữ hay không
                bool isShiftPressed = (GetAsyncKeyState(VK_SHIFT) & 0x8000) != 0;

                if (isShiftPressed) {
                    // Shift + PrintScreen -> Bật/Dừng quay video
                    PostMessage(g_hMainWnd, WM_DSNAP_TRIGGER, TRIGGER_VIDEO, 0);
                } else {
                    // PrintScreen đơn thuần -> Chụp ảnh
                    PostMessage(g_hMainWnd, WM_DSNAP_TRIGGER, TRIGGER_PHOTO, 0);
                }

                // QUAN TRỌNG: Trả về 1 để nuốt luôn sự kiện phím,
                // ngăn không cho Windows hoặc app khác (OneDrive, ShareX cũ,...) nhận diện
                return 1;
            }
        }
    }
    // Chuyển tiếp các phím khác cho hệ điều hành xử lý bình thường
    return CallNextHookEx(g_hKeyboardHook, nCode, wParam, lParam);
}

// Trong hàm xử lý cửa sổ chính (WndProc)
LRESULT CALLBACK WndProc(HWND hWnd, UINT uMsg, WPARAM wParam, LPARAM lParam) {
    switch (uMsg) {
    case WM_DSNAP_TRIGGER:
        if (wParam == TRIGGER_PHOTO) {
            TriggerOverlay((HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE), MODE_PHOTO);
        } else if (wParam == TRIGGER_VIDEO) {
            if (g_isRecording) StopRecording();
            else TriggerOverlay((HINSTANCE)GetWindowLongPtr(hWnd, GWLP_HINSTANCE),
MODE_VIDEO);
        }
        return 0;
```

```
    case WM_DESTROY:
        if (g_hKeyboardHook) {
            UnhookWindowsHookEx(g_hKeyboardHook);
            g_hKeyboardHook = NULL;
        }
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hWnd, uMsg, wParam, lParam);
}

// Khởi tạo trong WinMain
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int) {
    // ... Khởi tạo cửa sổ background ...
    g_hMainWnd = hWnd;

    // Đăng ký Hook toàn hệ thống (không cần dll ngoài)
    g_hKeyboardHook = SetWindowsHookEx(
        WH_KEYBOARD_LL,
        LowLevelKeyboardProc,
        hInstance,
        0 // 0 = hook toàn cục tất cả thread
    );

    // Vòng lặp GetMessage/DispatchMessage giữ cho hook hoạt động
    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return 0;
}
```

2 lưu ý thực tế trên Windows 11

1. Xung đột Windows Snipping Tool: Trên Windows 11, Microsoft có tùy chọn mặc định “Use the Print screen key to open screen capture”. Khi dùng WH_KEYBOARD_LL và trả về 1, hook của bạn vẫn chặn được hầu hết các trường hợp. Tuy nhiên, để sạch sẽ tuyệt đối, bạn nên tắt tùy chọn này trong Windows:
 • Settings $\rightarrow$ Accessibility $\rightarrow$ Keyboard $\rightarrow$ Tắt mục "Use the Print screen button to open screen capture".
2. Quyền hạn (UAC Elevation): Nếu có một cửa sổ đang chạy quyền Administrator (như Task Manager, CMD Admin), tiến trình dsnap.exe (chạy quyền User thường) sẽ bị Windows chặn không cho hook phím khi bạn đang focus vào cửa sổ Admin đó (cơ chế bảo mật UIPI của Windows). Để đè được phím trên mọi cửa sổ, dsnap.exe cũng cần chạy với quyền Administrator.